

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 14: ANÁLISE DE LATÊNCIA NA COMUNICAÇÃO MPI

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 15/09/2026.

1. EXPERIMENTO

Implemente um programa MPI com exatamente dois processos. O processo 0 deve enviar uma mensagem ao processo 1, que imediatamente responde com a mesma mensagem. Meça o tempo total de execução de múltiplas trocas consecutivas dessa mensagem, utilizando MPI_Wtime. Registre os tempos para diferentes tamanhos, desde mensagens pequenas (como 8 bytes) até mensagens maiores (como 1 MB ou mais). Analise-o graficamente o tempo em função do tamanho da mensagem e identifique os regimes onde a latência domina e onde a largura de banda se torna o fator principal.

2. CONCEITOS RELEVANTES

- **MPI**: permite que processos, possivelmente executando em computadores diferentes, cooperem em uma mesma aplicação por meio de comunicação explícita, como envio e recebimento de dados.
- **MPI_Init:** inicializa o ambiente MPI.
- **MPI_Finalize:** encerra o ambiente MPI ao final da execução.
- **MPI_COMM_WORLD:** comunicador padrão que agrupa todos os processos da execução.

- **MPI_Comm_rank:** obtém o identificador do processo atual, chamado de rank.
- **MPI_Comm_size:** obtém a quantidade total de processos MPI em execução.
- **MPI_Send:** envia uma mensagem para outro processo.
- **MPI_Recv:** recebe uma mensagem enviada por outro processo.
- **MPI_CHAR:** define que os dados da mensagem são caracteres, com um byte por elemento.
- **MPI_Barrier:** sincroniza os processos, fazendo cada um esperar até que todos cheguem ao mesmo ponto.
- **MPI_Wtime:** retorna um valor de tempo usado para medir a duração das trocas de mensagens.
- **MPI_STATUS_IGNORE:** informa que os detalhes da mensagem recebida não precisam ser armazenados ou consultados.

3. MÁQUINA UTILIZADA

A execução foi realizada no cluster do NPAD utilizando a partição “intel-512”. O script Slurm solicitou dois nós computacionais, com uma tarefa MPI em cada nó (--nodes=2, --ntasks=2 e --ntasks-per-node=1). Assim, os dois processos do teste ping-pong foram executados em máquinas distintas, permitindo medir o custo da comunicação pela rede entre nós do cluster. O tempo máximo reservado para a execução foi de 10 minutos.

Figura 1: Código do Slurm

```
#!/usr/bin/env bash
#SBATCH --job-name=tarefa14-mpi
#SBATCH --nodes=2
#SBATCH --ntasks=2
#SBATCH --ntasks-per-node=1
#SBATCH --time=00:10:00
#SBATCH --output=tarefa14-%j.out
#SBATCH --partition=intel-512

set -euo pipefail
cd "${SLURM_SUBMIT_DIR:-.}"
```

```
make
srun --ntasks=2 ./ping_pong > resultados.csv
```

4. CÓDIGO

O código implementa um teste de comunicação MPI no modelo ping-pong entre dois processos. Primeiro, ele inicializa o ambiente MPI e verifica se a execução possui exatamente dois processos. Em seguida, aloca um buffer de até 1 MiB, que será utilizado para enviar e receber mensagens.

Para cada tamanho de mensagem, de 8 bytes até 1 MiB, o programa sincroniza os processos com MPI_Barrier e inicia a medição do tempo. O processo 0 envia a mensagem ao processo 1 e espera sua resposta; o processo 1 recebe a mensagem e a devolve imediatamente. Essa troca é repetida 10.000 vezes para cada tamanho de buffer.

Figura 2: Código do experimento

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum {
    tamanhoInicial = 8,
    tamanhoMaximo = 1024 * 1024,
    numeroTrocas = 10000
};

int main(int argc, char **argv) {
    int rank, numeroProcessos;
    char *mensagem = malloc(tamanhoMaximo);

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &numeroProcessos);

    if (numeroProcessos != 2) {
```

```
    if (rank == 0)
        fprintf(stderr, "Use exatamente 2 processos.\n");
    MPI_Finalize();
    return 1;
  }

  // Preenche a mensagem com 'a' para completar o tamanho máximo,
  // garantindo que a memória esteja alocada e inicializada.
  memset(mensagem, 'a', tamanhoMaximo);
  if (rank == 0)
    puts("tamanhoBytes,tempoTotalSegundos");

  for (int tamanho = tamanhoInicial; tamanho <= tamanhoMaximo; tamanho *= 2) {
    MPI_Barrier(MPI_COMM_WORLD);
    double inicio = MPI_Wtime();

    for (int i = 0; i < numeroTrocas; i++) {
      if (rank == 0) {
        MPI_Send(
          mensagem, // Envia a mensagem para o processo 1
          tamanho, // Tamanho da mensagem em bytes
          MPI_CHAR, // Tipo de dado da mensagem
          1, // Destino da mensagem (processo 1)
          0, // Tag da mensagem
          MPI_COMM_WORLD // Comunicador global
        );
        MPI_Recv(
          mensagem, // Recebe a mensagem do processo 1
          tamanho, // Tamanho da mensagem em bytes
          MPI_CHAR, // Tipo de dado da mensagem
          1, // Origem da mensagem (processo 1)
          0, // Tag da mensagem
          MPI_COMM_WORLD, // Comunicador global
          MPI_STATUS_IGNORE // Status da operação (não é usado)
        );
      } else {
        MPI_Recv(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        MPI_Send(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
```

```
        }
    }

    if (rank == 0)
        printf("%d,%.9f\n", tamanho, MPI_Wtime() - inicio);
}

free(mensagem);
MPI_Finalize();
return 0;
}
```

5. RESULTADOS

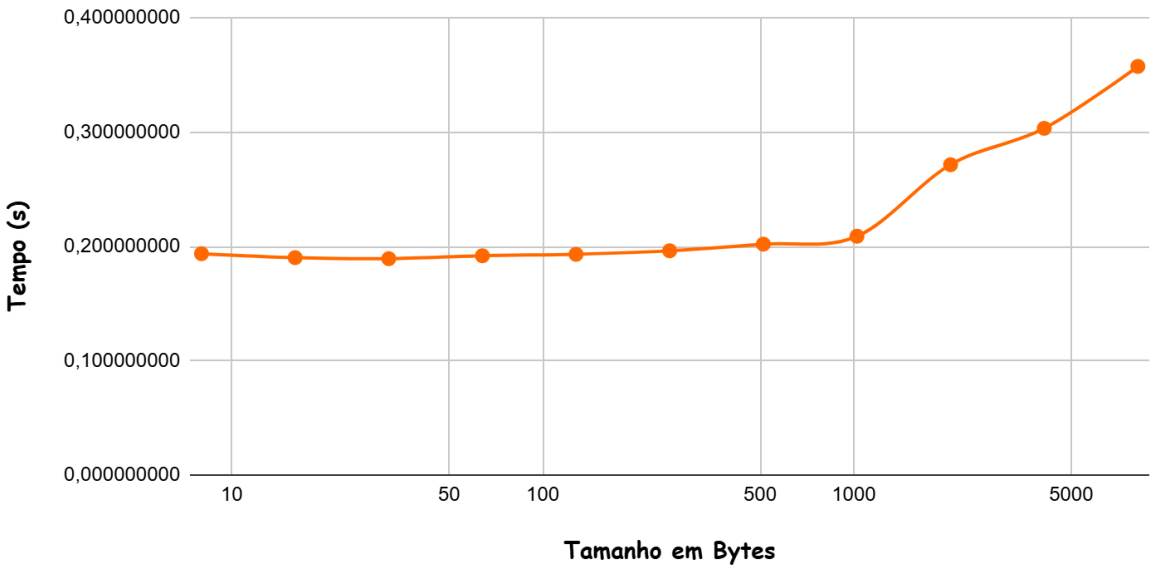
Abaixo, é possível visualizar o resultado de desempenho de compilação dos códigos.

Tabela 1: Tempo de comunicação variando tamanho da mensagem

Tamanho	Tempo (s)
8	0,193681620
16	0,190182374
32	0,189344632
64	0,191904582
128	0,193164546
256	0,196091381
512	0,201991863
1024	0,208853964
2048	0,271563485
4096	0,303264117
8192	0,357488372
16384	0,509367120
32768	0,757867715
65536	1,472399694
131072	2,074439752
262144	2,936345681

524288	4,543580954
1048576	7,656000241

Figura 3: Tempo de comunicação variando tamanho da mensagem



Ao analisar o gráfico e a tabela, podemos perceber que até cerca de 1 KiB, o tempo é quase constante, pois a latência e a sobrecarga de iniciar a comunicação dominam o custo, independentemente do tamanho do buffer.

Entre 2KiB e 64KiB ocorre uma transição em que o tamanho da mensagem começa a influenciar o tempo.

Acima de aproximadamente 64KiB, especialmente próximo de 512KiB, a largura de banda se torna o principal fator. O aumento ocorre porque buffers maiores exigem mais tempo para serem copiados e transmitidos pela rede, possivelmente também envolvendo mudanças no protocolo MPI e maior pressão sobre a memória e os caches.